ETSI GR ENI 060 V4.1.1 (2026-08)



GROUP REPORT

Experiential Networked Intelligence (ENI); Study on Model Training for Network AI Agents

Disclaimer

The present document has been produced and approved by the Experiential Networked Intelligence (ENI) ETSI Industry Specification Group (ISG) and represents the views of those members who participated in this ISG.
It does not necessarily represent the views of the entire ETSI membership.

Reference
DGR/ENI-0060_SMTNAIA

Keywords
AI-native, core network, GenAI

ETSI

650 Route des Lucioles
F-06921 Sophia Antipolis Cedex - FRANCE

Tel.: +33 4 92 94 42 00 Fax: +33 4 93 65 47 16

Siret N° 348 623 562 00017 - APE 7112B
Association à but non lucratif enregistrée à la
Sous-Préfecture de Grasse (06) N° w061004871

Important notice

The present document can be downloaded from the
ETSI Search & Browse Standards application.

The present document may be made available in electronic versions and/or in print. The content of any electronic and/or print versions of the present document shall not be modified without the prior written authorization of ETSI. In case of any existing or perceived difference in contents between such versions and/or in print, the prevailing version of an ETSI deliverable is the one made publicly available in PDF format on ETSI deliver repository.

Users should be aware that the present document may be revised or have its status changed, this information is available in the Milestones listing.

If you find errors in the present document, please send your comments to the relevant service listed under Committee Support Staff.

If you find a security vulnerability in the present document, please report it through our Coordinated Vulnerability Disclosure (CVD) program.

Notice of disclaimer & limitation of liability

The information provided in the present deliverable is directed solely to professionals who have the appropriate degree of experience to understand and interpret its content in accordance with generally accepted engineering or other professional standard and applicable regulations.
No recommendation as to products and services or vendors is made or should be implied.
No representation or warranty is made that this deliverable is technically accurate or sufficient or conforms to any law and/or governmental rule and/or regulation and further, no representation or warranty is made of merchantability or fitness for any particular purpose or against infringement of intellectual property rights.
In no event shall ETSI be held liable for loss of profits or any other incidental or consequential damages.

Any software contained in this deliverable is provided "AS IS" with no warranties, express or implied, including but not limited to, the warranties of merchantability, fitness for a particular purpose and non-infringement of intellectual property rights and ETSI shall not be held liable in any event for any damages whatsoever (including, without limitation, damages for loss of profits, business interruption, loss of information, or any other pecuniary loss) arising out of or related to the use of or inability to use the software.

Copyright Notification

No part of this document may be reproduced in any form, by any means and in any media, without the prior written authorization of ETSI and except as expressly permitted below.
By way of exception and when the document is a normative deliverable (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)), ETSI authorizes to reproduce and incorporate into products, services and technical documentation only those extracts (e.g. templates) that are strictly necessary for the technical implementation of the normative deliverable, to ensure compliance with the latter.
Nothing in this notice shall be construed as limiting any mandatory exceptions to copyright provided by applicable law.

© ETSI 2026.
All rights reserved.

Contents

Intellectual Property Rights 5
Foreword 5
Modal verbs terminology 5
1 Scope 6
2 References 6
2.1 Normative references 6
2.2 Informative references 6
3 Definition of terms, symbols and abbreviations 7
3.1 Terms 7
3.2 Symbols 8
3.3 Abbreviations 8
4 Introduction 9
4.1 Overview 9
4.2 Foundational Model Selection 9
4.3 Domain Adaptation via Continued Pre-training 10
4.4 Instruction Following via Supervised Fine-Tuning (SFT) 10
4.5 Post-training for Policy, Safety, and Behavioural Alignment 11
4.6 Evaluation and Verification 12
4.7 Deployment Optimization for Inference 12
5 Technical Roadmap for Model Training of Network AI Agents 12
5.1 Challenges and Requirements 12
5.1.1 Introduction 12
5.1.2 Domain Knowledge Gap 12
5.1.3 Data Preparation and Task Adaptation 13
5.1.4 Reliability, Safety, and Security 13
5.1.5 Evaluation System 13
5.2 Model Training Process for Network AI Agents 13
5.2.1 Introduction 13
5.2.2 Continued Pre-Training for Domain Adaptation 14
5.2.3 Supervised Adaptation for Task-Specific Behaviour 14
5.2.4 Policy-Aware Alignment 14
5.2.5 Deployment-Oriented Optimization 14
5.2.6 Evaluation and Verification 14
6 Continued Pre-training for Network AI Agents 15
6.1 Description 15
6.2 Training Data Preparation 15
6.3 Continued Pre-training Process 16
7 Deliberative Alignment in Telecommunications 17
7.1 Introduction 17
7.2 Rules and guidelines 17
7.3 Supervised Fine-Tuning 18
7.3.1 Introduction 18
7.3.2 Instruction Data Preparation 18
7.3.3 Instruction Fine-Tuning 20
7.4 Reinforcement learning 22
7.4.1 Introduction 22
7.4.2 Reward Model 22
7.4.3 Experiments 23
8 Model Compression and Efficient Adaptation 24
8.1 Introduction 24
8.2 Quantization 25
8.3 Pruning 26

8.4 Knowledge Distillation....27
8.4.1 Introduction....27
8.4.2 Whitebox Knowledge Distillation....27
8.4.3 Blackbox Knowledge Distillation....27
8.5 Low-Rank Adaptation and QLoRA....27
9 Conclusions and Recommendations....28
9.1 Summary....28
9.2 Future Work....28
Annex A: Illustration of Semantic Correlation Graph Construction....30
History....31

Intellectual Property Rights

Essential patents

IPRs essential or potentially essential to normative deliverables (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)) may have been declared to ETSI. The declarations pertaining to these essential IPRs, if any, are publicly available for **ETSI members and non-members**, and can be found in ETSI SR 000 314: *"Intellectual Property Rights (IPRs); Essential, or potentially Essential, IPRs notified to ETSI in respect of ETSI standards"*, which is available from the ETSI Secretariat. Latest updates are available on the ETSI IPR online database.

Pursuant to the ETSI Directives including the ETSI IPR Policy, no investigation regarding the essentiality of IPRs, including IPR searches, has been carried out by ETSI. No guarantee can be given as to the existence of other IPRs not referenced in ETSI SR 000 314 (or the updates on the ETSI Web server) which are, or may be, or may become, essential to the present document.

Trademarks

The present document may include trademarks and/or tradenames which are asserted and/or registered by their owners. ETSI claims no ownership of these except for any which are indicated as being the property of ETSI, and conveys no right to use or reproduce any trademark and/or tradename. Mention of those trademarks in the present document does not constitute an endorsement by ETSI of products, services or organizations associated with those trademarks.

DECT™, **PLUGTESTS™**, **UMTS™** and the ETSI logo are trademarks of ETSI registered for the benefit of its Members. **3GPP™**, **LTE™** and **5G™** logo are trademarks of ETSI registered for the benefit of its Members and of the 3GPP Organizational Partners. **oneM2M™** logo is a trademark of ETSI registered for the benefit of its Members and of the oneM2M Partners. **GSM**® and the GSM logo are trademarks registered and owned by the GSM Association.

Foreword

This Group Report (GR) has been produced by ETSI Industry Specification Group (ISG) Experiential Networked Intelligence (ENI).

Modal verbs terminology

In the present document "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the ETSI Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in ETSI deliverables except when used in direct citation.

1 Scope

The present document addresses the training and pre-deployment evaluation of AI models used within Network AI Agents in the ENI System context. It focuses on the selection, domain adaptation, fine-tuning, alignment, compression, and offline evaluation of model components intended for use by such agents in telecommunications scenarios. Specifically, the present document explores how decisions taken regarding model training strategies affect the resulting performance metrics of the model, and identifies key decision boundaries that need to be considered when training a model for a specific task. Consequently, identifying an appropriate pipeline of training methods, as well as a corresponding set of performance evaluation mechanisms, is of high importance.

The focus of the present document is primarily on the model training process for network AI agents. Several features of agent-based network architectures, such as the interfaces between agents and network functions, inter-agent coordination procedures, cross-operator interoperability mechanisms, or post-deployment lifecycle functions are out of scope of the present document and are not considered. Additionally, the present document focuses on pre-deployment aspects of model training and evaluation, and does not address certain (important) post-deployment aspects of the AI model lifecycle, such as in-production evaluation or telemetry.

The purpose of the present document is to identify technical decision points and evaluation considerations for model training, and to clarify how those choices affect the suitability of AI models for Network AI Agent roles within ENI.

2 References

2.1 Normative references

Normative references are not applicable in the present document.

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

NOTE: While any hyperlinks included in this clause were valid at the time of publication, ETSI cannot guarantee their long term validity.

The following referenced documents are not necessary for the application of the present document but they assist the user with regard to a particular subject area.

[i.1] ETSI GR ENI 056 (V4.1.1): "Experiential Networked Intelligence (ENI); Study on Multi-Agent Frameworks for Next-Generation Core Networks".

[i.2] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain and J. Gao: "Large Language Models: A Survey," arXiv preprint, vol. arXiv:2402.06196, 2024.

[i.3] H. Gisserot-Boukhlef, N. Boizard, M. Faysse, D. M. Alves, E. Malherbe, A. F. Martins, C. Hudelot and P. Colombo: "Should We Still Pretrain Encoders with Masked Language Modeling?", arXiv preprint, vol. arXiv:2507.00994, 2025.

[i.4] A. Radford, et al.: "Improving Language Understanding by Generative Pre-Training", June 2018.

[i.5] J. Gu, et al.: "A Survey on LLM-as-a-Judge", November 2024.

[i.6] Ç. Yıldız, N. K. Ravichandran, N. Sharma, M. Bethge and B. Ermis: "Investigating continual pretraining in large language models: Insights and implications", arXiv preprint, vol. arXiv:2402.17400, 2024.

[i.7] H. Shi, Z. Xu, H. Wang, W. Qin, W. Wang, Y. Wang, Z. Wang, S. Ebrahimi and H. Wang: "Continual learning of large language models: A comprehensive survey", ACM Computing Surveys, vol. 58, no. 5, pp. 1-42, 2025.

[i.8] J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai and Q. V. Le: "Finetuned Language Models are Zero-Shot Learners", in International Conference on Learning Representations, 2022.

[i.9] B. Peng, C. Li, P. He, M. Galley and J. Gao: "Instruction tuning with gpt-4", arXiv preprint, vol. arXiv:2304.03277, 2023.

[i.10] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray and J. Schulman: "Training language models to follow instructions with human feedback", Advances in neural information processing systems, vol. 35, pp. 27730-27744, 2022.

[i.11] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon and C. Finn: "Direct preference optimization: Your language model is secretly a reward model", Advances in neural information processing systems, vol. 36, pp. 53728-53741, 2023.

[i.12] TeleQnA open benchmark dataset.

[i.13] E. Hu, et al.: "LoRA: Low-Rank Adaptation of Large Language Models", ICLR 2022.

[i.14] Hugging Face LoRA Conceptual Guide.

[i.15] Z. Shao and et al: "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models", arXiv prepint, no. arXiv:2402.03300, 2024.

[i.16] Hugging Face, Inc.: "TRL - Transformer Reinforcement Learning", 2020.

[i.17] S. Rajbhandari, J. Rasley, O. Ruwase and Y. He: "Zero: Memory optimizations toward training trillion parameter models", in IEEE™ International conference for high performance computing, networking, storage and analysis, 2020.

[i.18] ETSI GS ENI 005 (V4.1.1): "Experiential Networked Intelligence (ENI); ENI System Architecture".

[i.19] Hugging Face PEFT library.

[i.20] Anthropic: "Constitutional Classifiers: Defending against universal jailbreaks", February 2025.

3 Definition of terms, symbols and abbreviations

3.1 Terms

For the purposes of the present document, the terms given in ETSI GR ENI 056 [i.1], ETSI GS ENI 005 [i.18] and the following apply:

AI Agent: autonomous system that is able to interact with its environment to collect data, learn from past experiences, and subsequently use these experiences to perform specific tasks

deliberative alignment: safety-alignment method in which a model explicitly retrieves and reasons over relevant policy or safety rules before generating a response

NOTE: Deliberative Alignment is commonly implemented using explicit reasoning processes (e.g., chain-of-thought reasoning) where the model recalls and applies relevant specifications before generating an output.

planning agent: AI Agent within the network responsible for generating task plans

NOTE: It receives service requests and decomposes them into a set of ordered, executable tasks.

3.2 Symbols

Void.

3.3 Abbreviations

For the purposes of the present document, the following abbreviations apply:

3GPP	3rd Generation Partnership Project
6G	6th Generation of Mobile Communications Technology
AI	Artificial Intelligence
AMF	Access Management Function
API	Application Programming Interface
BLER	Block Error Rate
CLM	Causal Language Modelling
CoT	Chain of Thought
COT	Chain of Thought
CPT	Continued Pre-Training
CPU	Computer Processing Unit
DPO	Direct Preference Optimization
E2E	End-to-end
FP32	32-bit Floating Point
GRPO	Group Relative Policy Optimization
KD	Knowledge Distillation
KL	Kullback-Leibler
KPI	Key Performance Indicator
KV	Key-Value (Cache)
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MLM	Masked Language Modelling
MLP	Multi-Layer Perceptron
NLP	Natural Language Processing
NPU	Neural Processing Unit
NR	New Radio
PDU	Protocol Data Unit
PEFT	Parameter-Efficient Fine-Tuning
PLM	Pre-trained Language Model
PTQ	Post-Training Quantization
QAT	Quantization-Aware Training
QLoRA	Quantization with Low-Rank Adaptation
QoS	Quality of Service
RAN WG1-3	Working Groups 1-3 of the 3GPP Technical Specification Group on RAN
RFC	Request for Comments
RL	Reinforcement Learning
RLHF	Reinforcement Learning from Human Feedback
RSRP	Reference Signal Received Power
SaaS	Software as a Service
SCG	Semantic Correlation Graph
SFT	Supervised Fine-Tuning
SINR	Signal to Interference Noise Ratio
SLA	Service Level Agreement
UE	User Equipment
UPF	User Plane Function

4 Introduction

4.1 Overview

The focus of ENI is to define an adaptable and context-aware network architecture which leverages techniques from Artificial Intelligence (AI) to better meet the needs of users and operators. One such technique is the embedding of AI Agents as entities within the architecture - that is, network components which may provide feedback or take actions based on inference of an associated AI model. AI Agents are expected to play a key role in ensuring a high quality of the service provided by the network to users, and therefore the performance characteristics of the models underpinning these agents (for example in terms of latency or inference accuracy) are of particular importance. With this in mind, the present document aims to highlight the challenges and technical approaches to training AI models for applications in telecommunications.

The inclusion of AI Agents in an agentic design for Next-Generation Core Networks has been proposed and discussed previously in [i.1]. However, an analysis of the methodology for preparing (i.e. selecting or fine-tuning) the AI models contained within Network AI Agents remains as yet unexplored, especially in the context of meeting the specific demands of telecom services.

The recent proliferation of Large Language Models (LLMs) has marked a significant turning point in the field of artificial intelligence, fundamentally altering the landscape of Natural Language Processing (NLP) and enabling a wide array of applications from advanced conversational agents to complex scientific discovery. A pivotal shift occurred with the introduction of Pre-trained Language Models (PLMs), which established the "pre-training and fine-tuning" paradigm. These models, distinguished by their massive scale in parameters and training data, exhibit emergent abilities such as in-context learning, instruction following, and multi-step reasoning, making them foundational components for general-purpose AI agents [i.2]. While foundational pre-training on vast corpora provides a broad linguistic and factual base for language models, it is merely the first step. The research community's focus is increasingly shifting toward a sophisticated suite of "post-training" techniques designed to refine, specialize, and align these base models. The following clauses introduce and describe the key stages that constitute the modern LLM training lifecycle.

4.2 Foundational Model Selection

The selection of a foundational model for subsequent domain-specific fine-tuning is of critical importance for the development of Network AI Agents. This phase needs to rigorously evaluate model performance along three dimensions:

- **Architectural suitability**;
- **Efficiency** at scale (particularly important for telco-grade deployment); and
- **Security**, including model provenance.

The choice of the model architecture and pre-training objective fundamentally affects the model's suitability for agentic workflows. Two common pre-training objectives are Causal Language Modeling (CLM) and Masked Language Modeling (MLM) [i.3]. For planning- and generation-oriented agents that need to produce open-ended sequential outputs, models trained with a causal or autoregressive decoding objective are often more suitable than pure masked-language encoders, because they are optimized for left-to-right sequence generation. This aligns with tasks such as generating multi-step decisions, action plans, and policy-aware response sequences. However, this does not imply that only decoder-only architectures are suitable. Encoder-decoder and other Transformer variants also support such tasks, provided that their decoding mechanism supports autoregressive generation and that they meet the controllability, latency, and deployment requirements of the target Network AI Agent [i.4].

In addition, the scale of the model needs to be calibrated against the operational constraints of the telco domain. While general-purpose AI benefits from universal scaling laws - where increasing model parameter counts broadly yields enhanced capabilities - telecommunication networks impose strict constraints regarding ultra-low latency and high reliability. In practice, deploying prohibitively large models for use in real-time control plane functions is highly impractical due to inference bottlenecks. Empirical evidence suggests that mid-sized foundational models (typically in the range of 7B to 35B parameters) offer the optimal trade-off. When fine-tuned on high-quality and dense telecom-specific datasets (e.g. 3GPP specifications, signalling logs, proprietary operational procedures, etc.), these models typically achieve domain-specific performance comparable to larger models, while offering the key advantages of reduced latency and computational budgets, as required for deployment in the edge or core network.

Lastly, the decision between internal development and external procurement is dictated by stringent data sovereignty and security mandates. The traffic of the core network encompasses information from highly sensitive domains such as user privacy information and data from critical infrastructure, making the reliance on external, API-based commercial models (SaaS solutions) a prohibitive security risk. Therefore, models are likely to be deployed internally (on operator premises). However, the foundational model itself does not need to be trained internally from scratch. The most pragmatic and cost-effective strategy involves procuring state-of-the-art open-weight or commercial foundational models, and subsequently performing all domain-specific fine-tuning, alignment, and deployment entirely within the operator's secure, air-gapped network boundary. This hybrid approach ensures both access to cutting-edge AI architectures and absolute control over proprietary telecom data.

4.3 Domain Adaptation via Continued Pre-training

While foundational pre-training equips LLMs with a vast repository of general knowledge and linguistic abilities, the performance of the resulting models are sometimes suboptimal in specialized, high-stakes domains. Niche or highly technical fields often rely on specific terminologies, nuanced contexts, and a body of knowledge that is typically underrepresented in general web text. Continued Pre-training (CPT), also known as continual learning or domain-adaptive pre-training, is a crucial post-training phase designed to address this gap by adapting a general-purpose foundational model to a specific domain [i.6].

CPT employs standard self-supervised objectives over a curated, domain-specific dataset. A critical design question in this phase is determining the sufficient volume of domain data required for effective adaptation. Empirical observations suggest that massive scale is of less importance than data density and quality. This suggests that a suitable approach is fine-tuning the base model on a highly curated corpus of training data, from sources such as 3GPP specifications, IETF RFCs, vendor manuals, and sanitized signalling logs.

However, it is noted that CPT is sometimes a delicate balancing act. When employed without care, CPT sometimes induces catastrophic forgetting - a phenomenon where the model's weights shift to minimize domain-specific loss at the expense of previously encoded general knowledge. In the context of Network AI Agents, general capabilities (particularly logical reasoning and instruction following) remain essential for complex orchestration and workflow generation. Therefore, defining an acceptable forgetting boundary is paramount. A strict zero-forgetting policy is computationally impractical and restricts domain specialization. Instead, an acceptable forgetting rate on broad, encyclopaedic knowledge benchmarks is generally tolerated. In summary, use of CPT requires careful management to ensure that, even as domain-specific accuracy increases substantially, the model's core logical and coding reasoning capabilities remain stable.

To enforce this acceptable threshold and mitigate severe degradation, techniques such as hyperparameter tuning and data mixing are employed [i.7]. Ultimately, validating the success of this domain adaptation requires a rigorous, agent-centric dual-track evaluation framework. The first track evaluates intent-driven task decomposition: it measures the model's capacity to accurately parse ambiguous natural language intents from users, decompose these complex objectives into atomic network operations, and autonomously orchestrate executable, multi-step workflows. The second concurrent track assesses domain compliance by auditing generated workflows and configuration scripts against relevant core network standards (e.g. 3GPP standards documents), operator policies, and valid signalling constraints.

4.4 Instruction Following via Supervised Fine-Tuning (SFT)

After foundational pre-training and optionally domain adaptation, an LLM possesses extensive knowledge but lacks the ability to apply it in a useful or safe manner. For example, a base model trained solely on next-token prediction does not inherently understand how to follow instructions, engage in conversation, or adhere to human ethical norms. Its raw outputs, reflecting the unfiltered nature of the web-scale training data used in the foundational pre-training phase, sometimes is unhelpful, factually incorrect, biased, or even harmful. To address this limitation, a critical series of post-training steps are applied to transform the raw predictive engine of the language model into a reliable and cooperative AI agent. Together, these steps are typically referred to as the alignment phase of language model training.

The first and most fundamental step of the alignment process is to teach the model to follow instructions. The format of the instructions is closely related to the tasks performed by a given agent. For example, an agent responsible for task planning is able to receive natural language requests from users, while agents operating closer to the network function layer instead receive semi-structured or policy-constrained requests associated with specific services or procedures.

Instruction following is achieved through a SFT procedure which adapts a pre-trained base model by further training it on a high-quality, labelled dataset of instruction-response pairs [i.8]. This supervised learning process adjusts the model's weights to teach it the general skill of interpreting a user's prompt and generating an appropriate, well-formatted response. The success of instruction tuning is heavily dependent on the quality of the input training data. These instruction datasets are typically sourced in one of two ways:

1) **Human-Crafted Datasets:** These are created by human annotators who write diverse prompts and expert-level responses. This approach results in high quality training data, but is expensive and time-consuming.

2) **Model-Generated Datasets:** To address the increased cost and inefficiency of manual curation, scalable approaches often leverage advanced LLMs to synthetically generate instruction-response pairs for later review, filtering, or augmentation. This is more accurately understood as synthetic data generation guided by a teacher model than as classical teacher-student distillation [i.9]. Deploying this strategy within telecom environments requires careful attention to provenance, reproducibility, licensing constraints, data-boundary control, and the risk of transferring bias or hallucinated patterns into the supervised fine-tuning corpus:

    a) **Compliance and Sovereignty Risks:** Use of externally hosted proprietary models sometimes conflicts with operator data-boundary requirements, confidentiality obligations, or licensing restrictions where sensitive network information is involved.

    b) **Algorithmic and Bias Risks:** Extensive reliance on machine-generated data may propagate latent bias, systematic errors, or hallucinated procedural patterns into the resulting model.

Examples of key failure modes of SFT-trained AI models in the telecommunications context include:

- Sensitivity to data distribution shift arising from changing traffic, mobility, interference, and multi-timescale network dynamics, which may cause initially small errors to propagate through later planning steps.
- Overfitting to operator-specific policies, heuristics, or legacy tuning rules where the training data are narrowly derived from a single operational environment.
- Under-representation of rare but safety-critical events in training data, such as blackouts, severe congestion, or equipment failures leading to weak performance on infrequent but operationally important cases.

4.5 Post-training for Policy, Safety, and Behavioural Alignment

While SFT-based instruction tuning teaches language models the format of a good response, it does not explicitly teach it to prefer helpful and safe responses over plausible but incorrect or harmful ones. Post-training alignment refines model behaviour with respect to operational policies, security constraints, privacy boundaries, and service safety objectives. Different techniques connect model behaviour to written policy and expert judgement in different ways, including rule-guided fine-tuning, preference optimization, and deliberative alignment. In telecommunications, alignment is relevant not only to refusal behaviour but also to bounded action generation, constraint awareness, traceability, and operator oversight.

For reference, Reinforcement Learning from Human Feedback (RLHF) [i.10] is a widely used technique that incorporates human preferences into the training loop using a reward signal. It is a multi-stage process that ultimately refines the weights of the SFT model.

In contrast, Direct Preference Optimization (DPO) [i.11] is a more recent technique that pursues a similar alignment objective without requiring an explicit reward model or a separate reinforcement learning stage. The core insight of DPO is that the KL-constrained preference optimization objective is reformulated as a direct optimization problem over preference data. In practice, DPO increases the relative likelihood of preferred responses and decreases the relative likelihood of rejected responses, while remaining implicitly regularized by a reference model. For Network AI Agents, the effectiveness of DPO depends on the quality and representativeness of the underlying preference data, and the resulting model still needs to be evaluated for technical correctness, policy compliance, and operational suitability in the target deployment context.

4.6 Evaluation and Verification

Evaluation in the network domain extends beyond generic language benchmarks. Relevant evidence includes domain question-answering accuracy, task completion quality, policy compliance, hallucination rate, robustness to incomplete or adversarial inputs, latency under deployment conditions, and expert assessment of technical correctness. Evaluation datasets and scoring procedures also need clear provenance and separation between training, validation, and test material.

Evaluation is role-dependent and deployment-aware. In addition to generic task accuracy, assessment of a Network AI Agent needs to consider whether outputs are technically correct, policy-compliant, operationally bounded, and suitable for the target execution environment. Evaluation therefore includes failure-mode analysis, robustness to ambiguous or incomplete inputs, sensitivity to changes in network context, and expert review of high-impact outputs. Where deployment is intended, latency, memory footprint, and hardware compatibility are assessed under representative execution conditions rather than inferred from offline benchmark results alone.

4.7 Deployment Optimization for Inference

Following pre-training, adaptation, and alignment, the resulting LLM is highly capable but often too large or too computationally demanding for practical deployment. State-of-the-art models sometimes contain billions (or more) of parameters, and typically result in significant memory and inference resources. This often makes such models expensive and difficult to deploy in real-world, production environments with constrained latency, compute, or energy budgets. Deployment optimization is therefore relevant to reducing memory footprint, inference cost, and response latency while preserving acceptable task performance.

Relevant optimization methods include quantization, pruning, knowledge distillation, and low-rank methods. These techniques differ in their effects on memory footprint, inference latency, training cost, and model quality, and their suitability depends on the role of the Network AI Agent and the characteristics of the target hardware platform. In particular, some methods are primarily relevant to deployment-time compression, whereas others, such as QLoRA and related low-rank adaptation approaches, are primarily relevant to reducing the cost of model adaptation rather than directly compressing the final deployed model artifact.

In the networking domain, the selection of a deployment-optimization method needs to take into account the latency budget of the target agent, the relevant hardware targets, and the acceptable trade-off between efficiency and task performance. The detailed characteristics and trade-offs of these methods are discussed in clause 8.

5 Technical Roadmap for Model Training of Network AI Agents

5.1 Challenges and Requirements

5.1.1 Introduction

Training models for use within Network AI Agents differs from general-domain model adaptation because the target environment is technically specialized, operationally constrained, and closely coupled to external systems. In telecommunications settings, the model is expected to operate over structured terminology, protocol procedures, performance indicators, configuration semantics, and policy constraints that are only partially represented in general-purpose training corpora.

5.1.2 Domain Knowledge Gap

General-purpose models lack expertise in telecommunications knowledge required for reliable reasoning over network procedures and operational context. This gap includes specialized terminology (e.g. "beamforming" and "handover delay"), protocol structures, signalling procedures, KPI semantics and metrics (e.g. RSRP, SINR, BLER), inter-function dependencies, and the procedural logic embedded in standards and operational documents.

For this reason, domain adaptation is not just a matter of vocabulary extension. It also requires improvement in the model's internal representation of relationships among entities, procedures, constraints, and performance objectives that arise in telecommunications workflows as opposed to simple syntactical and lexical matching. In addition, the model also needs to comprehend structured telecommunications artefacts including complex signalling flowcharts and protocol state transitions, and Protocol Data Unit (PDU) structures described in 3GPP standards and vendor implementation documents.

A practical implication is that a model used within a Network AI Agent needs to be able to reason over domain-specific procedures such as PDU session establishment, including both the signalling interactions and the underlying resource allocation logic involved. Without sufficient domain adaptation, a model sometimes generates outputs that appear plausible but are technically unsound due to its inability to understand the technical meaning behind the input content. Hence, a primary challenge is how to incorporate high-quality telecommunications knowledge into the model. This is not merely domain fine-tuning but rather requires the large-scale introduction of high-quality communication-specific corpora, including 3GPP standard documents, technical white papers, operator and vendor manuals and network reports, academic papers, and expert knowledge bases.

5.1.3 Data Preparation and Task Adaptation

Even after domain adaptation, the model used within a Network AI Agent still needs to be adapted to the specific tasks assigned to that agent. These tasks include (but are not limited to) intent interpretation, planning, decomposition into executable subtasks, retrieval formulation, tool selection, diagnostic explanation, or policy-aware workflow generation.

A central difficulty at this stage is the preparation of suitable training and evaluation data. Telecommunications data and documentation are heterogeneous in source, structure, modality, and quality, and therefore substantial effort is often needed to normalize, align, annotate, filter, and structure them into datasets that are usable for model training and assessment. This often requires the integration of data from multiple sources and modalities, together with the construction of a sufficiently consistent representation for fine-tuning and evaluation.

5.1.4 Reliability, Safety, and Security

The operational tolerance for model error is lower in network environments than in many general-purpose conversational settings. Outputs that are plausible but technically incorrect often lead to invalid plans, unsafe actions, unsuitable recommendations, privacy violations, policy non-compliance, or degradation of service continuity. In some cases, such errors often produce SLA violations and associated operational or commercial impact.

Accordingly, model training for Network AI Agents is closely related to requirements for policy compliance, bounded action generation, confidentiality preservation, and resistance to unsafe or poorly grounded outputs. This does not imply that model training alone guarantees meeting desired reliability, safety, and security objectives, but it does mean that these objectives materially influence dataset design, alignment procedures, evaluation criteria, and deployment choices.

5.1.5 Evaluation System

Evaluation in this context extends beyond generic language benchmarks. A practically relevant evaluation framework includes (but is not limited to) assessment of domain knowledge, protocol understanding, quality of planning, factual consistency, policy compliance, robustness to ambiguous or incomplete inputs, and expert judgement of technical correctness and operational usefulness.

Evaluation is also task-dependent. A model intended for planning, diagnosis, optimization, or intent handling typically require different datasets, metrics, and human review procedures, depending on the assisted-system role and operational context in which they are deployed.

5.2 Model Training Process for Network AI Agents

5.2.1 Introduction

In light of the above challenges, the training process for a model used within a Network AI Agent can be viewed as a staged process. The stages described below are intended as a technical roadmap that links general-purpose language-model capabilities to telecommunications-specific use.

5.2.2 Continued Pre-Training for Domain Adaptation

The first stage adapts a selected base model to the telecommunications domain through continued pre-training on curated corpora relevant to the target environment. Depending on the use case, these corpora typically include standards-related documents, operator procedures, technical reports, manuals, and other domain-specific materials. The purpose of this stage is to improve the model's handling of telecommunications terminology, protocol relationships, procedural structure, and domain context. In effect, it reduces the gap between general linguistic competence and the domain-specific understanding needed for network-related reasoning.

5.2.3 Supervised Adaptation for Task-Specific Behaviour

After domain adaptation, the model is further adapted to the tasks expected within the agent's operational scope. This stage commonly uses supervised fine-tuning on task-oriented examples whose prompts, outputs, and, where appropriate, intermediate task structures reflect the intended interfaces and decision points of the assisted system. During supervised adaptation, the model learns to map instructions to outputs and actions that are aligned with system-specific constraints. A domain-specific training dataset is therefore needed to guide how network-related tasks are represented and resolved. These training data typically include instructions, expected outputs, and in some cases, optional intermediate reasoning and annotations that help represent decomposition steps or other task structure. It is also beneficial to include negative examples (e.g. unsupported or unsafe requests) to teach the model appropriate refusal behaviour.

5.2.4 Policy-Aware Alignment

A further stage refines model behaviour with respect to operational rules, safety objectives, privacy boundaries, and other policy constraints. In telecommunications settings, this stage is important because technically fluent outputs are not necessarily operationally acceptable.

Alignment is often implemented through combinations of rule-guided supervised training, preference-based optimization, deliberative alignment, or related techniques. The goal is to improve the model's ability to produce responses and plans that remain within defined behavioural and operational bounds.

5.2.5 Deployment-Oriented Optimization

Once the model has been adapted and aligned, deployment-oriented optimization is applied to reduce resource consumption and improve runtime characteristics. Relevant methods include quantization, pruning, knowledge distillation, and other compression or efficiency techniques. Some low-rank methods may also be relevant in deployment workflows, although low-rank adaptation is often more directly related to reducing the cost of model adaptation than to compressing the final deployed model artifact.

The importance of this stage depends on deployment context. The trade-offs differ across centralized platforms, edge environments, and other resource-constrained settings in which latency, memory footprint, and energy use directly affect deployment feasibility.

5.2.6 Evaluation and Verification

Evaluation and verification methods establish whether the trained model is suitable for its intended role within the Network AI Agent. This includes both quantitative and expert-based assessment of technical correctness, task performance, policy compliance, robustness, and operational relevance. The outcome of this stage informs iterative refinement of earlier stages, including corpus curation, task-data design, alignment procedures, and optimization choices. Hence, the training roadmap is not purely linear, but supports repeated adjustment based on observed deficiencies, changing requirements, and deployment needs.

The following clauses examine these stages in greater detail in the context of a planning-oriented Network AI Agent, which is able to interpret user intents and generate workflows to fulfil user requirements.

6 Continued Pre-training for Network AI Agents

6.1 Description

This clause describes the continued pre-training stage used to adapt the primary model of a Network AI Agent to telecommunications-specific material before later stages of task alignment, behavioural refinement, and deployment optimization are applied. In the present study, continued pre-training is treated as a domain-adaptation step whose purpose is to improve the model's handling of telecommunications terminology, procedural structure, and technical context, rather than as a standalone demonstration of complete agent behaviour.

Continued pre-training is a post-foundational training stage in which a pre-trained language model is further trained on a curated domain corpus in order to improve its representation of the target domain. In the context of Network AI Agents, this stage is used to reduce the gap between general-purpose language competence and the domain-specific understanding needed to interpret telecommunications concepts, protocols, procedures, and performance indicators. In the present document, continued pre-training is therefore treated as a domain-adaptation step rather than as a standalone demonstration of complete agent behaviour.

In the present document, the selected base model is DeepSeek-R1-Distill-Qwen-7B, which is published as a distilled model based on the Qwen 2.5 family. The role of continued pre-training in this setting is to improve the model's familiarity with telecommunications-specific vocabulary, document structure, and procedural context so that later stages of supervised fine-tuning and alignment are able to operate on a stronger domain-adapted basis.

Continued pre-training is therefore relevant to the overall capability of the agent, but it is only one part of the broader model-training lifecycle described in clauses 5, 7 and 8. Overall agent behaviour remains influenced by other elements of the assisted system, including control logic, tools, memory, interfaces, and applied policies.

6.2 Training Data Preparation

The effectiveness of continued pre-training depends strongly on the quality, relevance, of the domain corpus used for adaptation. For telecommunications use cases, the corpus may include standards-related text, technical reports, operator procedures, engineering documentation, and other material that reflects the terminology, procedures, and task context of the intended deployment environment. In the present study, the training material was assembled to cover both established and emerging network architectures.

The effectiveness of continued pre-training depends strongly on the quality, relevance, and traceability of the domain corpus used for adaptation. For telecommunications use cases, the corpus typically includes (but is not limited to) standards-related text, technical reports, operator procedures, engineering documentation, and other material that reflects the terminology and task context of the intended deployment environment. In the present document, the training material was systematically assembled to cover both established and emerging network architectures relevant to the target evaluation scope. The acquisition strategy focused on two primary source categories:

- Standardized Technical Specifications: A curated selection of 3GPP materials, with a concentrated focus on Radio Access Network (RAN WG1-3) and System Architecture (SA2, SA5) sources. This helps the model to capture authoritative procedures for New Radio (NR) interfaces, mobility management, and QoS policy frameworks.

- Pre-standardization 6G Research: To extend the model's predictive and analytical capabilities toward future technologies, the corpus includes early 6G architectural studies, specifically targeting End-to-End (E2E) AI service procedures and next-generation terminology. This broadens coverage of forward-looking terminology and design patterns.

The preparation process includes extraction, normalization, segmentation, metadata tagging, provenance retention, and deduplication so that the resulting corpus is more suitable for model training. More specifically, key preparation steps include:

- Extraction and Normalization: A specialized pipeline of text from heterogeneous sources was employed to strip non-semantic elements (e.g. headers, footers, and redundant metadata) from heterogeneous sources (PDF, HTML). Technical abbreviations and nomenclature were normalized to ensure conceptual consistency across different document versions.

- Structural Categorization: Content was segmented into logical functional blocks, such as procedural definitions and architectural requirements, and enriched with provenance metadata (e.g. 3GPP Working Group origin, licensing status, confidentiality constraints, and version information). This structured approach facilitates downstream filtering and improves the model's ability to distinguish between and updates to different protocol layers.

- Quality Assurance and Deduplication: Beyond basic grammatical validation, the corpus underwent semantic clustering to identify and merge near-duplicate content frequently found in overlapping technical reports. This step prevents gradient bias toward over-represented procedures and ensures a balanced distribution of technical knowledge.

These preparation steps serve several purposes. They improve textual consistency, reduce repeated or low-value content, and preserve useful technical structure for later training stages. They also help align the training material with the procedural and semantic patterns that the model is expected to recognize in telecommunications tasks.

From a lifecycle perspective, corpus preparation is not limited to collection alone. It also includes decisions about source selection, provenance retention, quality control, and version handling, since these factors influence both the behaviour of the adapted model and the interpretability of later evaluation results.

6.3 Continued Pre-training Process

After preparation of the domain corpus, the selected base model undergoes additional training on telecommunications-specific material in order to improve domain adaptation. In this clause, the process is understood as continued pre-training of the model component used within the agent, rather than as training of the complete agent framework.

The base model used in the present study was DeepSeek-R1-Distill-Qwen-7B, a 7B-parameter model that is published as a distilled model based on the Qwen 2.5 family. The role of continued pre-training in this setting is to improve the model's familiarity with telecommunications-specific vocabulary, document structure, and procedural context so that later stages of supervised fine-tuning and alignment are able to build on a stronger domain-adapted basis.

The continued pre-training stage in this study employs an autoregressive (causal) language modelling objective. During this process, the model is trained to predict the next token in a sequence over the curated corpus described in clause 6.2. This is done by minimizing the negative log-likelihood to predict the next token in a sequence using a cross-entropy loss function. This enables the model to learn the conditional probability distributions of technical sequences. The immediate objective of this stage is to improve the model's representation of telecommunications concepts, protocol relationships, signalling context, and technical terminology that are not sufficiently captured by general-purpose pre-training alone. Implementation choices such as learning-rate control, precision strategy, and data mixing are relevant to limiting excessive degradation of retained general capabilities during adaptation.

To maintain the model's foundational reasoning capabilities and avoid catastrophic forgetting, a low learning rate (e.g. 1e-5) and a mixed-precision (BF16/FP32) strategy are utilized. This adaptation process allows the model handling of:

- Technical Nomenclature: Precise usage and conceptual mapping of 3GPP-specific terms and acronyms, ensuring the model distinguishes between general and domain-specific meanings of common terms.

- Procedural Syntax: The logical flow of network signalling, state transition sequences, and protocol hierarchies, enabling the model to "anticipate" the next step in a standard procedure.

- Contextual Reasoning: The semantic and functional relationships between network entities (e.g. UE, AMF, UPF), enabling the model to recognize network entities and their relationships within complex deployment and mobility scenarios.

The immediate objective of this stage is to improve the model's representation of telecommunications concepts, protocol relationships, signalling context, and technical terminology that are not sufficiently captured by general-purpose pre-training alone. In practical terms, this supports later tasks such as protocol interpretation, technical question answering, plan drafting, and structured response generation, while not by itself establishing full operational reliability or safe autonomous action.

To assess post-training performance, an evaluation dataset was constructed using the open benchmark dataset TeleQnA [i.12] augmented with an additional 6G-oriented question-answer dataset referred to as 6GQnA. The 6GQnA dataset consists of question-answer pairs covering telecommunications-related concepts, terminology, and standards-related topics. For 6GQnA, an initial set of 2 500 question-answer pairs was first curated by domain experts. These reference question-answer pairs were then expanded upon by using an LLM for prompt-based generation of additional question-answer pairs, which were further validated by the domain experts to remove ambiguous or redundant entries. As part of the provenance control, the expert review also excluded entries with unsupported answers or more than one plausible answer. The final 6GQnA evaluation set was kept separate from the continued-pre-training corpus and was not used for continued pre-training, post-training, to reduce evaluation leakage and preserve the validity of the performance assessment.

Under the reported evaluation procedure, the model was assessed across three dimensions: correctness (factual accuracy), integrity (coherent domain language), and logic (relevance to protocol-specific queries). The continued pre-trained model demonstrated a significant performance gain over the baseline, with the aggregate score increasing from 56 % to 81 %, corresponding to a 25 percentage-point improvement on the benchmark used in this study. This result indicates improved benchmark performance after continued pre-training. This is interpreted as evidence of better domain adaptation within the reported evaluation setting rather than as proof of telecom-grade operational assurance.

Accordingly, the present document treats continued pre-training as an enabling stage that strengthens the domain basis of the model before subsequent stages of supervised fine-tuning, alignment, behavioural refinement, evaluation, and deployment optimization are applied. The following clauses examine those later stages in greater detail.

7 Deliberative Alignment in Telecommunications

7.1 Introduction

This clause describes the post-training stage used to adapt the primary model of a Network AI Agent to task-specific instructions, policy artefacts, and behavioural preferences in a telecommunications environment. In the present document, deliberative alignment is treated as a behavioural refinement stage applied after continued pre-training, with the objective of improving policy-aware planning and response generation for a planning-oriented agent.

The approach described here combines three elements:

1) rules and guidelines used as policy artefacts;

2) supervised fine-tuning on an instruction dataset derived from network tools and service scenarios; and

3) reinforcement learning based on reward signals related to policy compliance, planning quality, and judged reasoning quality.

In this sense, deliberative alignment contributes to the behaviour of the trained model, while overall agent behaviour remains dependent on the wider assisted-system context, including control logic, tool interfaces, memory, external validation mechanisms, and operator oversight.

7.2 Rules and guidelines

In the present document, rules and guidelines are treated as two distinct forms of behavioural steering for the model. Their roles are related but not identical, and the distinction is important for interpreting the later supervised fine-tuning and reinforcement-learning stages.

Within a Network AI Agent, these artefacts serve as policy inputs to the model-alignment stage rather than as a complete substitute for external enforcement. Their role is to improve the probability that the model produces outputs that are compatible with telecommunications policies and planning expectations, but they do not remove the need for surrounding control mechanisms, validation logic, or operator review.

Rules describe behaviours that are treated as either allowed or not allowed within the planning context. They are intended to function as hard behavioural constraints within the aligned model objective; however, they do not by themselves replace external enforcement or validation. When a user request cannot be satisfied without violating an applicable rule, the expected model response is rejection, denial, or safe deferral of the request rather than circumvention of the constraint.

The rules illustrated in the present study cover several types of restrictions, including exclusion of subtasks unrelated to user intent, avoidance of dependence on unavailable or unconfigured physical facilities, rejection of privacy-sensitive or security-sensitive requests, and avoidance of actions that could damage infrastructure, degrade other users' service, or create broader safety risks. These examples are best interpreted as illustrative policy fragments for the planning use case rather than as a complete telecommunications policy model.

Guidelines are better understood as advisory information intended to steer the agent toward network designs or plans with desirable properties, without imposing the same kind of hard constraint as rules. Accordingly, the aligned model is expected to prefer guidelines when constructing a plan, while remaining within the applicable rules when trade-offs arise.

In the present document, guidelines are organized by service category and are used to structure the decomposition of a user request into relevant subtasks. The sensing example included in the draft illustrates this approach through steps such as access-rights checking, selection and configuration of sensing resources, identification of needed capabilities, and deployment of those capabilities as a service workflow.

An alternative approach is to provide relevant policies or regulations directly in the prompt at deployment time, as described in [i.20]. In the present document, deliberative alignment is treated instead as a training-based mechanism, whereby model weights are trained with policies and regulations in mind, as opposed to prompt-time policy injection.

Comparative efficiency, reliability, and operational trade-offs between prompt-time policy injection and training-based alignment are outside the scope of this clause.

7.3 Supervised Fine-Tuning

7.3.1 Introduction

After continued pre-training, supervised fine-tuning is used to adapt the model to the planning tasks and policy-aware response patterns relevant to the target Network AI Agent. In the present study, this stage is centred on a planning-oriented use case in which the model interprets a user request, reasons over relevant constraints, and produces a structured set of subtasks or service steps.

The purposes of this stage are to improve task-specific planning behaviour and to make the model's responses more consistent with the rules and guidelines introduced in clause 7.2. The resulting model is intended to produce outputs that are less generic and more explicitly conditioned by service constraints, permissions, and deployment considerations.

7.3.2 Instruction Data Preparation

The present document investigates the automatic generation of the instruction dataset for supervised fine-tuning rather than being authored entirely by domain experts. The generation process is a graph-based multi-stage pipeline that derives realistic user queries from the available tool ecosystem and then filters them for quality, relevance, and consistency with the specified rules and guidelines. The main stages of this pipeline are shown in Figure 7.3.2-1.

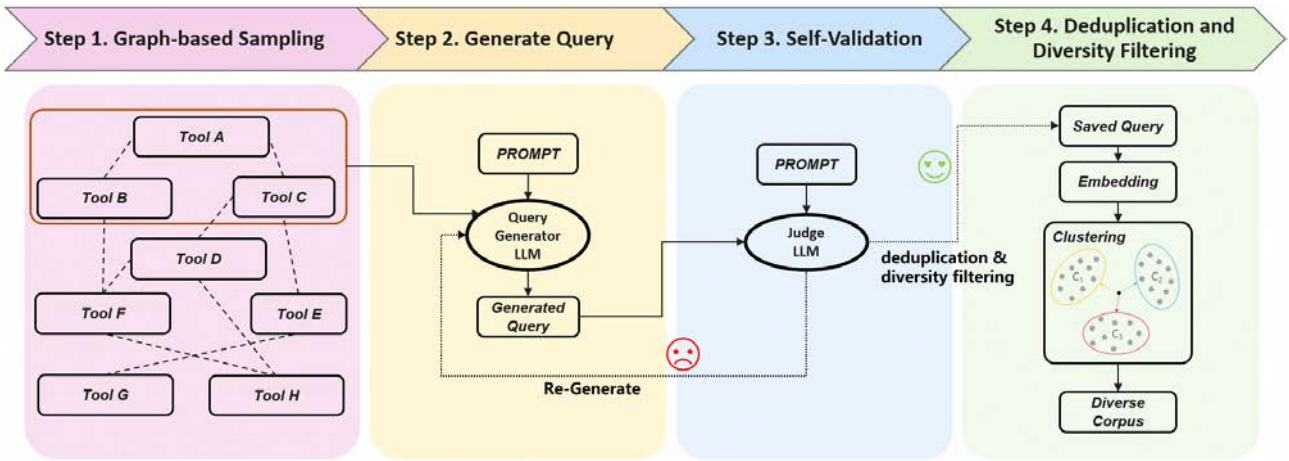


Figure 7.3.2-1: Main stages of the automatic generation and filtering pipeline used to construct the instruction dataset for supervised fine-tuning

Figure 7.3.2-1 illustrates the main stages of the automatic data-generation and filtering pipeline used to construct the instruction dataset for supervised fine-tuning. The process consists of three main stages, with a 4th data cleaning stage not represented on the figure for simplicity. The stages are as follows:

1) Constructing a tool chain semantic correlation graph from all available tools
2) Performing non-repetitive random walk sampling to obtain semantically coherent tool chains
3) Generating and validating detailed queries conditioned on the sampled chains
4) Deduplicating and diversity filtering of the accepted queries

This pipeline is intended to produce a large instruction dataset with broad coverage of tool combinations and service intents while remaining closer to the policy and task structure of the planning use case. The four stages are described below.

Stage 1: Tool Chain Correlation Graph Construction

The first stage constructs a Semantic Correlation Graph (SCG) in which nodes correspond to available tools and edges represent semantic relationships derived from tool descriptions, parameters, and outputs. These relationships are computed using embeddings and similarity measures over parameter-output, parameter-parameter, and tool-tool descriptions, with edges created when similarity exceeds a selected threshold. Examples of functionalities that are represented by tools include:

1) A tool to count the number and/or types of objects within a certain area
2) A tool to perform a statistical analysis on structured inputs
3) A tool to track a dynamically moving object

An illustration of the SCG construction process is provided in clause A.

Stage 2: Random Walk Sampling of Tool Chains

The second stage performs non-repetitive random walks over the graph to generate semantically coherent tool chains. These sampled chains provide the structural basis for downstream query generation, so that the resulting instructions reflect plausible combinations of network functions and service capabilities rather than arbitrary prompt text. Each walk captures a semantically coherent sequence of tools. This procedure is intended to provide wide coverage of possible tool combinations while preserving semantic coherence across the sampled chains.

Stage 3: Instruction Query Generation and Validation

The third stage uses one LLM to generate one or more detailed user queries conditioned on the sampled tool chains and example prompts. A second LLM then acts as a judge to assess the generated queries for quality, plausibility, and consistency with the specified rules and guidelines. Queries that fail validation are regenerated until acceptable instances are obtained or discarded according to the filtering criteria used in the study. In this study, the validation stage acts as a quality-control mechanism for dataset construction, not as proof that the accepted queries are complete, uniquely correct, or operationally safe in all deployment contexts.

Step 4: Deduplication and Diversity Checking

The final stage removes duplicates and near-duplicates and evaluates diversity across functional areas, query forms, and edge cases. The purpose of this stage is to reduce over-representation of similar instruction patterns and to improve coverage of the planning use case without relying only on raw query volume.

The purpose of this pipeline is to create a large instruction corpus with broad coverage of tool combinations and service intents while keeping the resulting dataset closer to the policy and task structure of the planning use case. However, the resulting corpus still depends on the quality of the available tool descriptions, the selected generation and filtering criteria, and any expert review applied during dataset preparation.

7.3.3 Instruction Fine-Tuning

The supervised fine-tuning stage in the present study uses a parameter-efficient approach based on LoRA within a Parameter-Efficient Fine-Tuning (PEFT) framework [i.19] and [i.14]. PEFT methods enable efficient adaptation of large pre-trained models to downstream applications by training only a small subset of additional parameters rather than all model weights. This significantly decreases the computational cost of training, and furthermore reduces storage demands for fine-tuned variants. Recent state-of-the-art PEFT techniques achieve performance comparable to fully fine-tuned models [i.13].

In this study, the Low-Rank Adaptation (LoRA) PEFT approach is used to adapt the DeepSeek-R1-Distill-Qwen-7B base model described in clause 6.3. LoRA augments selected weight matrices with low-rank trainable components while keeping the original model parameters fixed. During training, gradients are applied only to the low-rank matrices, which significantly reduces the number of trainable parameters and supports more efficient fine-tuning of the model.

The instruction dataset generated by the pipeline in clause 7.3.2 is used for finetuning. Training samples follow a triplet structure of the form {query, intermediate reasoning trace, output}. The purpose of this structure is to expose the model to task-decomposition patterns and policy-aware response structure during fine-tuning. The most important configuration parameters used during the training process are summarized in Table 7.3.3-1 for reproducibility of these results.

Table 7.3.3-1: Parameters used during the finetuning process, including both low-rank adaptation and general training parameters

LoRA Rank	16
LoRA Alpha	32
LoRA Dropout	0,05
Learning Rate	3e-4
Batch Size	32
Load In 8bit	True
Datatype	torch.float16

The training loss decreased sharply over the first 3 epochs of training, and continued to gradually decrease over the course of 10 more epochs, as illustrated in Figure 7.3.3-1. This pattern is consistent with convergence under the reported configuration, and is supposed to be interpreted together with the evaluation results described below rather than as an independent indicator of alignment quality.

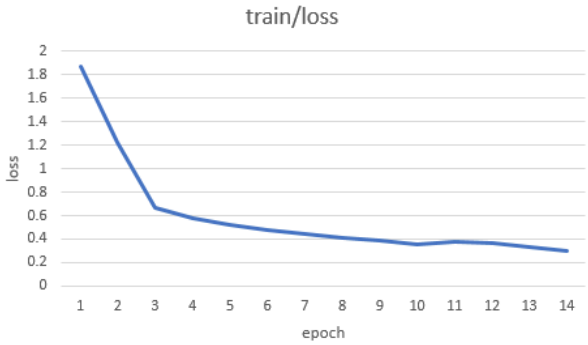


Figure 7.3.3-1: Illustration of the training process in terms of training loss

The impact of supervised fine-tuning on policy-aware behaviour was evaluated using an LLM-as-a-judge setup. For this evaluation, a compliance-oriented prompt of the following form was used:

Based on a performance evaluation of the finetuned model, the performance is improved both in terms of suitability for networking tasks and also in terms of respecting the relevant rules and guidelines, as described in clause 7.1. More specifically, the trained model, under the same prompt, even without explicitly providing rules and guidelines along with the user/3rd party intent, modifies its output to comply with rules and guidelines, such as ensuring access rights are respected. This indicates that the model has learned the relevant rules and guidelines during the instruction fine-tuning process.

The effectiveness of Supervised Fine-Tuning (SFT) in aligning the DEEPSEEK-R1-DISTILL-QWEN-7B model with the network policies and guidelines was evaluated using an LLM-as-a-judge framework. The evaluation utilized a specific compliance prompt:

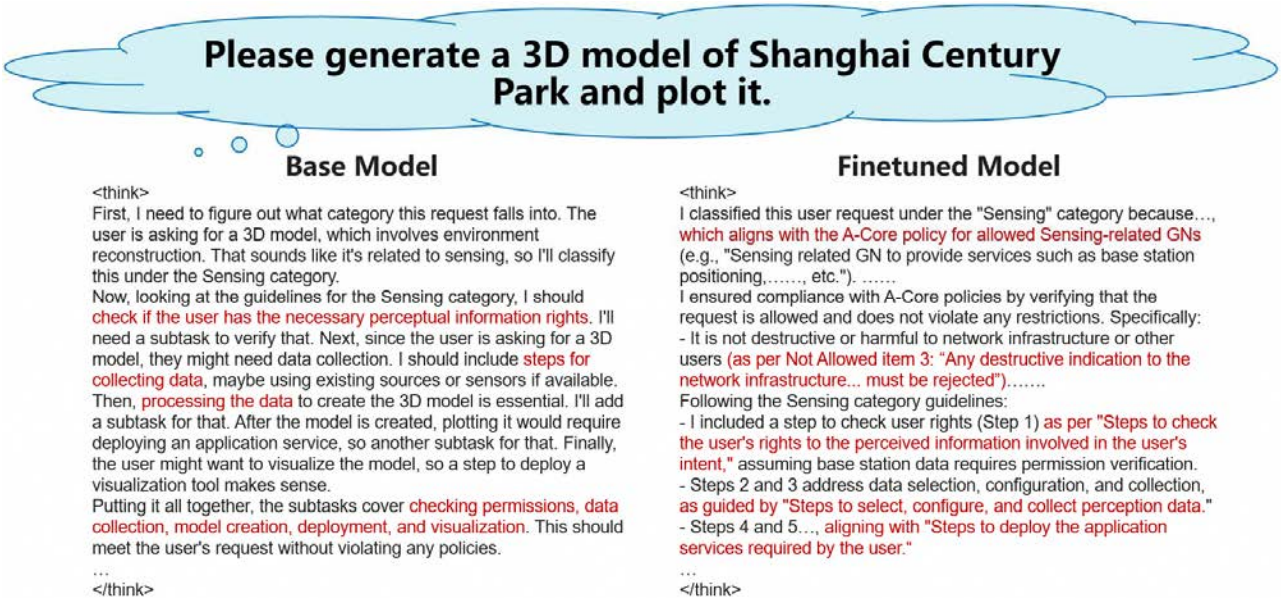
- *"Compliance: You should carefully confirm that the response complies with all of the following policies and guidelines: {}"*

Table 7.3.3-2: Compliance score achieved by base model and finetuned model on DEEPSEEK-R1-DISTILL-QWEN-7B

Model (DEEPSEEK-R1-DISTILL-QWEN-7B)	Base Model	Fine-tuned Model
Compliance Score	8,29	9,15

The evaluation compared the baseline performance of the base model (provided with explicit policy guidelines in the prompt) against the fine-tuned model (operating without explicit policy guidelines in the prompt). Quantitative results demonstrate that the policy-aware SFT yields a measurable enhancement in compliance performance. The mean compliance score increases from 8,29 for the base model to 9,15 for the fine-tuned version, representing a performance improvement exceeding 10 %. This evaluation indicates that, within the limits of the LLM-as-a-judge methodology and the test set used, fine-tuning on policy-aware instruction data improves judged compliance with the specified rules and guidelines. It provides evidence of improved judged policy adherence under the given prompt and benchmark conditions, rather than as a complete assessment of operational safety or regulatory conformity.

Figure 7.3.3-2 provides an illustrative example comparing Chain-of-Thought (CoT) outputs between the base and fine-tuned models for a specific user request. The side-by-side analysis shows how the fine-tuned model more consistently incorporates relevant rules and guidelines into its internal subtask decomposition and final response generation, including checks related to access rights and the handling of sensing-related resources.



NOTE: The red highlights illustrate notable points in the COT outputs where the finetuned model exhibits more specific behaviour compared to the base model.

Figure 7.3.3-2: An illustration of the model COT outputs for the base (left) and finetuned (right) models, given the same input query

7.4 Reinforcement learning

7.4.1 Introduction

Following supervised fine-tuning, reinforcement learning is applied to further refine planning behaviour under a reward signal derived from policy and task-quality judgements. The selected algorithm is Group Relative Policy Optimization (GRPO) [i.15] , in which multiple outputs are generated for the same prompt and compared within a group to compute relative advantages without a separate critic network.

The reinforcement learning stage uses the GRPO workflow shown in Figure 7.4.1-1. For each user query, the model generates multiple candidate outputs, each comprising an intermediate reasoning trace and a corresponding sequence of subtasks. These outputs are scored by the reward model and compared within the group to produce relative advantages, which are then used to update the policy. The optimization objective also includes a Kullback-Leibler (KL) divergence penalty so that the updated model remains anchored to a reference distribution [i.15].

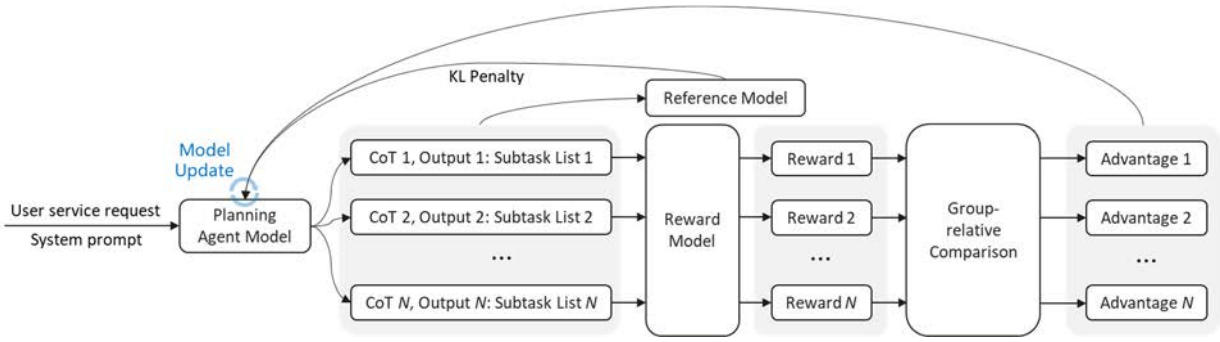


Figure 7.4.1-1: Schematic of a GRPO optimization loop used in the Reinforcement Learning Stage for Deliberative Alignment

The reinforcement learning stage is best understood as a preference-optimization step for open-ended planning tasks rather than as proof of uniquely correct network plans. This distinction matters because many telecommunications planning problems admit several acceptable decompositions or workflows rather than a single ground-truth answer.

7.4.2 Reward Model

A significant challenge in applying GRPO to networking tasks is that many service provisioning and network planning problems are often open-ended and lack a singular "ground-truth" solution. In this case, a high-capability model with knowledge of the various rules and guidelines relevant for the network agent acts as a domain expert (i.e. an LLM-as-a-Judge) [i.5]. This reward model is used as a proxy for expert judgement in order to score candidate outputs for reinforcement learning, and evaluate both the final answers and the model-generated reasoning trace.

For models intended for networking applications, reward design considers both adherence to operator rules and policies and broader planning quality. The latter model interprets user intent appropriately and decomposes the task into a sequence of executable subtasks.

For models intended for networking applications, reward design needs to consider both strict adherence to operator rules and policies and broader network planning proficiency. The latter includes whether the model interprets user intent appropriately and decomposes the task into a sequence of plausible and executable subtasks.

In the present study, the reward model evaluates model outputs across three dimensions:

1) **Policy and Guideline Compliance** (0-10 points): A measure of the extent to which the model output aligns with predefined network rules, safety constraints, and operational policies.

2) **Decision Effectiveness** (0-10 points): Evaluation of whether the produced plan or sequence of tasks is able to solve the specified network problem and meet service objectives (i.e. satisfies the user intent in an efficient manner).

3) **Reasoning Rigour** (0-10 points): Evaluation of the logical consistency, depth, and completeness of the generated trace.

The judge model receives the user request, the generated reasoning trace, the final subtask list, and the relevant policy documents as inputs. It then computes a composite score by averaging the rewards across these dimensions, providing a scalar signal to guide the policy optimization process. In GRPO terms, this scalar score serves as the reward input for policy optimization.

NOTE: This reward is a proxy for expert judgement rather than as a direct measure of operational correctness or deployment safety.

7.4.3 Experiments

The RL phase was implemented using the Hugging Face TRL (Transformer Reinforcement Learning) framework [i.16], with DeepSpeed ZeRO-2 used to support data-parallel training and memory optimization [i.17]. For each user query, four candidate outputs were generated from the model in order to support the group-based advantage estimation required by GRPO. The reported training configuration uses a per-device batch size of 2, combined with a gradient accumulation factor of 4 to achieve an effective global batch size suitable for stable convergence.

The evolution of the reward during the training and evaluation phases is shown in Figure 7.4.3-1 and Figure 7.4.3-2, respectively. As illustrated, the reported reward values increase during the early phase of training and then stabilize within a relatively high range under the adopted judge-based reward function. This behaviour is consistent with improvement on the optimization objective used in the study, but it is not by itself be interpreted as proof of broad operational generalization.

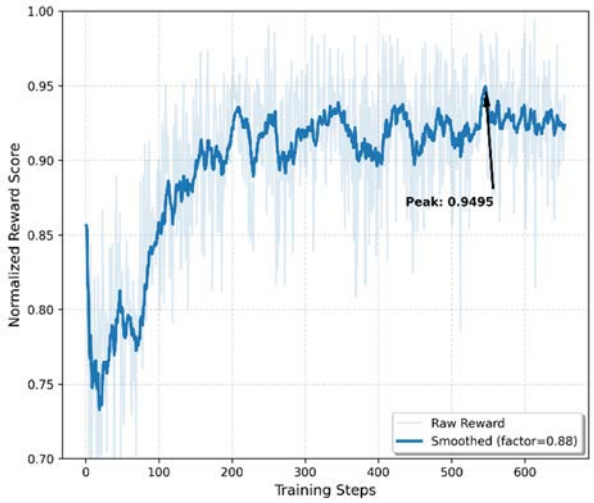


Figure 7.4.3-1: Evolution of Reward during Training

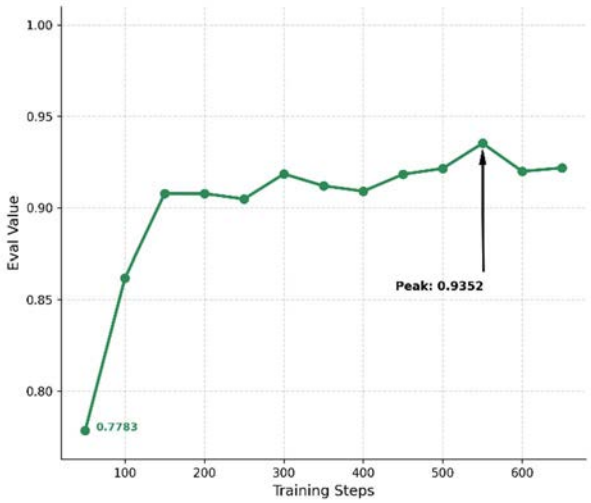


Figure 7.4.3-2: Evolution of Reward during Evaluation

A qualitative example is shown in Figure 7.4.3-3. Under the illustrated prompt, the post-RL model produces a more detailed reasoning trace and a more structured task decomposition than the comparison output. This example is useful as an illustration of the type of behaviour encouraged by the reward model, but it needs to be interpreted as a case example rather than as a substitute for systematic evaluation.

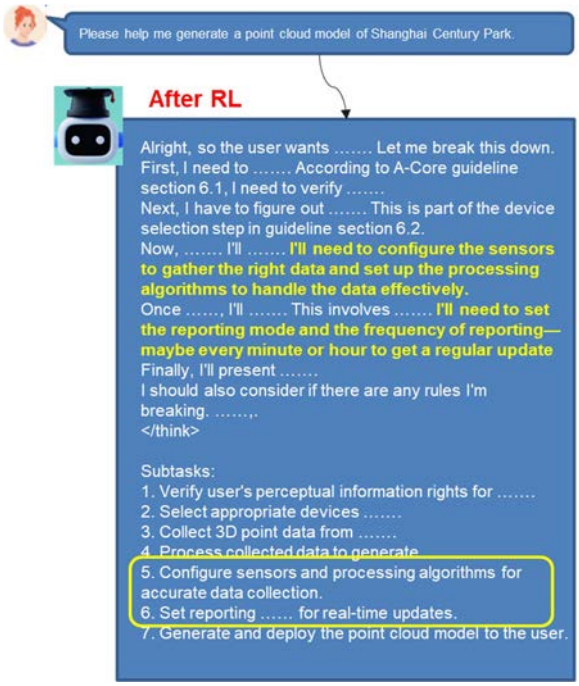


Figure 7.4.3-3: A Use Case including the CoT and Answer from RL-Trained Model

8 Model Compression and Efficient Adaptation

8.1 Introduction

This clause discusses methods for reducing deployment cost and fine-tuning resource requirements for Network AI Agents, including quantization, pruning, knowledge distillation, and low-rank adaptation methods.

In addition to deployment within the core network, Network AI Agents are also able to be deployed in resource-constrained edge nodes and, in some cases, directly on User Equipment (UE) in support of low-latency network management and closed-loop control. Systematic reduction of model memory footprint and computational cost is essential to ensure practical deployment.

In this clause, four families of techniques are discussed: quantization, pruning, knowledge distillation, and low-rank methods. These techniques address different parts of the deployment problem and involve different trade-offs between memory usage, latency, retraining cost, and task performance. Figure 8.1-1 provides a schematic overview of these four mechanism families.

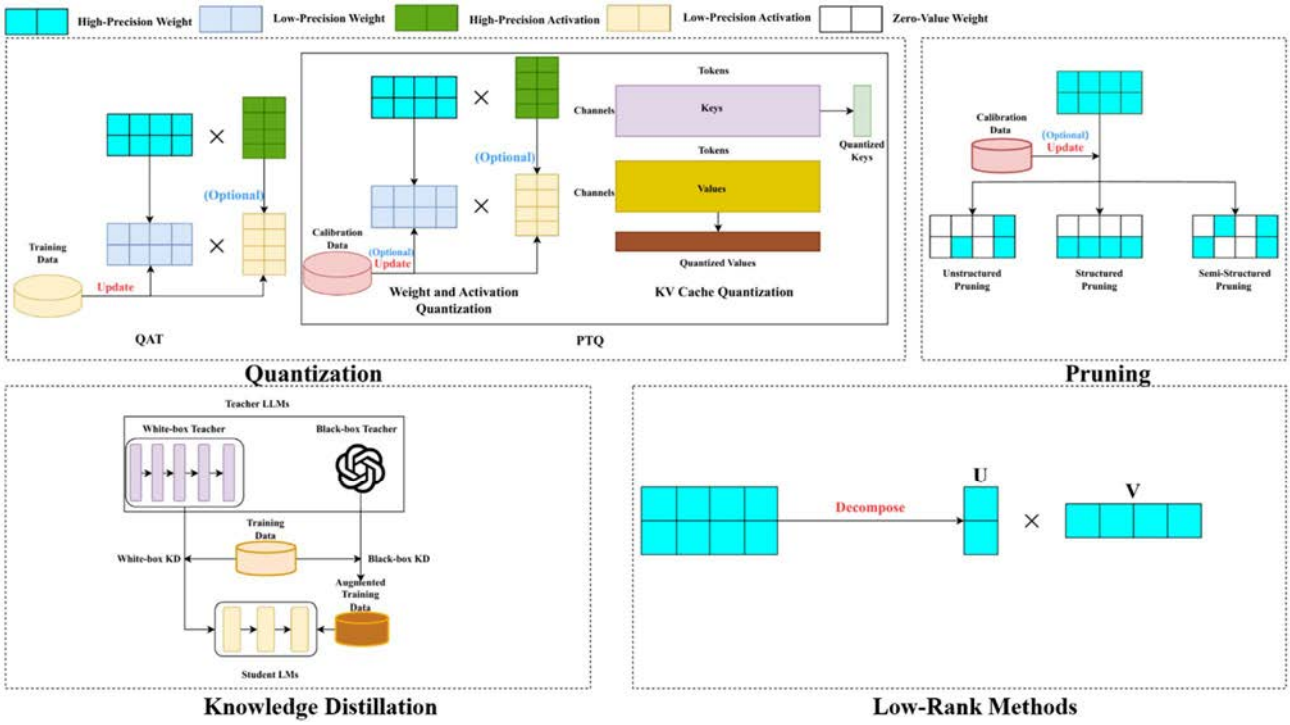


Figure 8.1-1: Schematic Overview of Generic Methods for Model Compression

8.2 Quantization

Model quantization involves approximating the continuous, high-precision values of a neural network with a discrete set of low-precision integers. For Network AI Agents, this reduction in numerical precision is able to significantly decrease storage requirements and accelerate inference on specialized hardware such as Neural Processing Units (NPUs).

The quantization process is applicable to several data structures relevant to model deployment:

- Weights and Activations: Quantizing weights reduces static storage, while quantizing activations reduces runtime memory consumption.
- Key-Value (KV) Cache: For LLM-based agents, KV cache quantization is implemented to support longer context windows and improve sequential inference efficiency.
- Gradients: In distributed training of network AI agents, gradient quantization reduces communication overhead across the network backbone, although this relates to training efficiency rather than inference-time compression.

Quantization commonly uses a mapping between floating-point values and lower-precision numerical representations defined by scale and offset parameters. The relationship between a floating-point value r and its quantized integer counterpart q is typically expressed as:

$$q = \text{clip}(\text{round}(\frac{r}{S} + Z), q_{\min}, q_{\max})$$

Where S denotes the scale factor and Z represents the zero-point offset. Symmetric and asymmetric schemes are both used in practice. Symmetric schemes (where $Z = 0$) simplify arithmetic and are often preferred in implementations targeting efficient inference, while asymmetric schemes fit skewed value distributions more closely. The *clip* function maintains all quantized values within the discrete set of low-precision integers bounded by $q_{\min}$ and $q_{\max}$.

A key design decision is whether quantization is applied after training or incorporated during training. Post-Training Quantization (PTQ) applies quantization to an already trained model and is attractive when deployment simplicity is the primary concern. Quantization-Aware Training (QAT) incorporates quantization effects during fine-tuning so that the model adapts to the target bit-width. In general, PTQ is simpler to apply, while QAT often preserves task performance better under aggressive low-bit quantization.

Table 8.2-1 is a generic illustrative example drawn from image-model quantization. It is included only to show the typical trade-off between compression and task performance; it is not a direct evaluation of a Network AI Agent or an LLM.

Table 8.2-1: Generic illustrative example of quantization trade-offs from an image-model setting

Accuracy Type	Strategy	CIFAR-100 Accuracy (%)	Performance Improvement (CPU)
FP32	Baseline	72,05	1,0x
INT8	PTQ	71,20	~2,0x
INT8	QAT	73,67	~2,0x

8.3 Pruning

Pruning refers to the systematic removal of model parameters or structures that contribute relatively little to task performance. In the context of Network AI Agents, the purpose of pruning is to reduce memory footprint and inference cost while preserving acceptable behaviour on the target networking tasks. Common pruning workflows include post-training pruning followed by recovery fine-tuning, pruning during training, and pruning followed by retraining from scratch.

There are generally three main approaches to the model pruning process:

- Train the model as normal, prune the trained model, then fine-tune it (also referred to as parameter recovery).
- Perform pruning during the training of the model, followed by fine-tuning of the trained model.
- Prune the base model prior to training, then retrain the pruned model from scratch.

Pruning methods are broadly divided into structured or unstructured pruning. Unstructured Pruning removes individual weights and typically produces sparse matrices. Although it is possible that this reduces the number of non-zero parameters, it typically does not translate into practical latency gains on standard deployment hardware unless specialized sparse execution support is available. Where deployment targets do not provide effective sparse-execution support, structured pruning is generally the more practical choice for achieving deployable latency and memory reductions. Structured pruning removes whole channels, heads, neurons, and/or layers and is therefore generally more suitable for deployment on conventional accelerator and CPU platforms used in telecommunications environments. For transformer-based models, pruning decisions needs to respect architectural dependencies, including residual connections, projection dimensions, attention heads, and MLP block compatibility. Dependency-aware pruning is therefore important to ensure that removing one structure does not invalidate tensor-shape consistency or degrade deployment.

The trade-off characteristics between structured and unstructured pruning are summarized in Table 8.3-1.

Table 8.3-1: Trade-off characteristics between unstructured and structured pruning

Feature	Unstructured Pruning	Structured Pruning
Granularity	Weight-level (fine-grained)	Channel/Filter-level (coarse-grained)
Hardware Friendliness	Low	Generally High
Inference Acceleration Potential	Depends on specialized hardware	More readily accelerated on conventional dense hardware
Accuracy Preservation Capability	Often high at moderate sparsity, but workload-dependent	Moderate (depends on pruning rate)
Matrix Form	Sparse matrix	Dense matrix (reduced in size)

The trade-off characteristics summarized in Table 8.3-1 are indicative rather than universal, since realized gains depend on pruning rate, model architecture, compiler/runtime support, and deployment hardware.

8.4 Knowledge Distillation

8.4.1 Introduction

Knowledge Distillation (KD) is a technique designed to transfer selected capabilities from a larger teacher model to a smaller student model. The purpose is to obtain a student model with lower deployment cost while preserving useful task performance. Two broad cases are considered here: whitebox distillation, in which internal teacher information is available, and blackbox distillation, in which only teacher-generated outputs are available. Key considerations for each form of knowledge distillation are discussed in the following clauses.

8.4.2 Whitebox Knowledge Distillation

Whitebox knowledge distillation uses internal information from the teacher model, such as output distributions or intermediate features, to guide student training. Whitebox methods are commonly grouped into three types:

- Offline distillation: The method used by most knowledge distillation algorithms. It mainly involves two stages:

  1) Pre-training the teacher model before distillation.

  2) Knowledge transfer via a suitable distillation algorithm.

  Teacher models usually have a large number of parameters and require long training times.

- Online distillation: Requires simultaneous updating of both teacher and student models. This approach mainly addresses situations where a large, high-performance teacher model is not readily available. However, existing methods often struggle to obtain such large, high-precision teacher models in an online environment.

- Self-distillation: A special case of online distillation where the teacher and student models share the same network architecture.

8.4.3 Blackbox Knowledge Distillation

In Blackbox Knowledge Distillation, internal parameters and logits from the teacher model are not available. Therefore, the teacher is used to generate a synthetic distillation dataset, which is then used to fine-tune the student model. The transferred capabilities include instruction following, in-context task generalization, and reasoning-style outputs such as chain-of-thought traces, depending on the supervision produced by the teacher. In-context learning, chain-of-thought supervision, and instruction-following data are better treated as categories of blackbox supervision signals than as distinct top-level distillation methods.

8.5 Low-Rank Adaptation and QLoRA

Low-rank methods require careful distinction in the present context. A low-rank factorization of model weights is able to be used as a compression technique, whereas Low-Rank Adaptation (LoRA) is primarily a parameter-efficient fine-tuning method. In the present study, the focus is on LoRA and QLoRA, which reduce the number of trainable parameters and the memory required during fine-tuning, rather than directly compressing the deployed base model by themselves.

The practical advantage of LoRA is that only a relatively small set of trainable parameters is stored and optimized during task adaptation. Prior work reports large reductions in trainable-parameter count and training-memory requirements relative to full fine-tuning, with little or no added inference latency when the adaptation is merged appropriately at deployment. However, these benefits are best understood as adaptation-efficiency gains rather than as direct proof of deployed-model compression.

9 Conclusions and Recommendations

9.1 Summary

The present document studies model-training methods for Network AI Agents in telecommunications. It identifies key challenges for this domain, including the need for specialized domain knowledge, the difficulty of preparing high-quality task data, and the requirement for high reliability, safety, and security in operational network environments.

To address these challenges, the present document describes a staged training pipeline comprising continued pre-training, deliberative alignment, and model optimization. Continued pre-training adapts a general model to telecommunications knowledge using domain-specific corpora, while deliberative alignment incorporates task guidance and policy-aware behaviour into model outputs.

For the Planning Agent example considered in the present document, the continued pre-training stage improved performance on the TeleQnA evaluation dataset from 56 % to 81 % using the base model DeepSeek-R1-Distill-Qwen-7B. This base model was pre-trained to incorporate specialized communication data, 3GPP standard documents, 6G research reports, and communication papers. Within the reported evaluation setting, this result provides evidence of improved domain adaptation rather than proof of telecom-grade operational assurance.

The present document further describes a deliberative-alignment workflow based on supervised fine-tuning and reinforcement learning, supported by automatically generated instruction data and policy-aware evaluation. An innovative graph-based multi-stage data generation and filtering framework is introduced in clause 7, which provides a method for constructing task-oriented training data for planning use cases, while the reported results indicate improved judged policy adherence under the evaluation conditions used in the study. This workflow is technically plausible for improving rule and guideline compliance, but further verification is still needed using broader task sets, stronger baselines, and more explicit operational safety criteria.

The present document also reviews model optimization methods relevant to practical deployment, including quantization, pruning, knowledge distillation, and low-rank adaptation methods. These methods involve different trade-offs between memory footprint, latency, training cost, and model quality, and their suitability depends on the deployment environment and task requirements of the target Network AI Agent.

Overall, the present report supports the conclusion that a staged training approach for Network AI Agents is technically plausible and useful, but still requires broader validation. In particular, future work prioritizes evaluation under representative telecom workloads, explicit safety and policy-conformance testing, reproducibility of results, and assessment of deployment constraints across core, edge, and UE environments.

9.2 Future Work

It is recommended that future versions of the present document focus on broader technical validation of the staged training pipeline under representative telecommunications workloads that includes planning, diagnosis, optimization, and intent-handling tasks across different assisted-system roles. It is further recommended that future work prioritize evaluation under explicit safety and policy-conformance testing, reproducibility of results, and assessment of deployment constraints across core, edge, and UE environments. Finally, the following topics merit further exploration in a future version of the present document:

- Representative evaluation datasets with clearer separation of training, validation, and test material, together with stronger provenance, traceability, and version control for domain corpora and synthetic instruction data.
- Evaluation of benchmark representativeness for real telecommunications tasks, including protocol reasoning, workflow generation, policy-aware planning, and rare but safety-critical operational scenarios.
- More explicit analysis of policy conflict handling, constraint prioritization, and consistency of model outputs when multiple operator rules, service objectives, or safety constraints interact.
- Stronger verification of LLM-as-a-judge methodologies, including judge calibration, agreement with human experts, sensitivity to prompt design, and robustness of reward signals used in reinforcement learning.

- Human-oversight mechanisms, including operator approval paths, intervention points, rollback procedures, refusal handling, and auditability of high-impact outputs in closed-loop environments.
- Runtime assurance beyond offline training and testing, including post-deployment monitoring, telemetry-driven evaluation, model drift detection, and mechanisms for controlled model update and re-validation.
- Interoperability with surrounding systems, including tool interfaces, memory components, external validation services, orchestration logic, and cross-agent coordination boundaries within the assisted system.
- Security and abuse-resistance evaluation, including adversarial prompting, prompt injection through tool outputs, confidentiality leakage, and unsafe action generation under ambiguous or malicious inputs.
- Comparative study of alignment methods, including supervised fine-tuning, reinforcement learning, rule-guided approaches, and preference-optimization methods such as DPO, under common telecom evaluation conditions.
- Deployment-oriented assessment of optimization methods on realistic hardware targets, including the effect of quantization, pruning, distillation, and low-rank methods on latency, memory footprint, and task quality under telecom execution constraints.
- Multi-operator and multi-domain generalization, including variation in local policy sets, vendor procedures, data distributions, and standards interpretations across deployment environments.
- Broader treatment of governance and compliance, including links between model-training practice, organizational controls, documentation obligations, and AI-management processes relevant to operational network environments.

Further contributions from academia and industry remain important for maturing these topics and for establishing a broader technical consensus on the training and evaluation of models used within Network AI Agents.

Annex A:
Illustration of Semantic Correlation Graph Construction

As shown in Figure A.1, the Semantic Correlation Graph (SCG) is a directed graph where tools are represented as vertices in a graph, where each tool is associated with a functional description, a set of input parameter descriptions, and a set of output specifications. Edges in the SCG denote inferred functional or sequential relationships between tools, derived from feature proximity in a latent embedding space. A candidate edge between two tools is established using a computation based on embeddings and semantic similarity, which is established under three correlation criteria:

- Parameter-Output (P-O) similarity: Measures the similarity between one tool's input parameter description and another tool's output specification, indicating potential data-flow compatibility.
- Parameter-Parameter (P-P) similarity: Measures the degree of similarity between the input parameter descriptions of two tools, indicating related data requirements or functional grouping.
- Tool-Tool (T-T) similarity: Measures the similarity between the high-level functional descriptions of two tools, reflecting topical or domain-level relevance.

More concretely, an edge $e_{i,j}$ in the SCG is established between the vertices v_i and v_j , which have tool descriptions parameter descriptions and output specifications denoted by t_i, p_i, and o_i, respectively. A correlation is hypothesized to exist if any of the above similarity measured exceeds a specified threshold τ, where the value of τ is empirically determined. This condition is expressed mathematically as follows:

$$e_{i,j} = \begin{cases} \exists p_i \subseteq v_i, o_j \subseteq v_j : \mathrm{SIM}(p_i, o_j) > \tau_1, \\ 1, \text{if } \exists p_i \subseteq v_i, p_j \subseteq v_j : \mathrm{SIM}(p_i, p_j) > \tau_2, \\ \exists t_i \subseteq v_i, t_j \subseteq v_j : \mathrm{SIM}(t_i, t_j) > \tau_3, \\ 0, \text{ otherwise}, \end{cases}$$

where SIM denotes the correlation measurement derived from the similarity of the corresponding embedding vectors.

An SCG was constructed for a sample network containing a broad mixture of network functions and tools. This resulted in a dense correlation graph of approximately 100 tools and 400 edges. Figure A.1 shows a simplified segment of that graph.

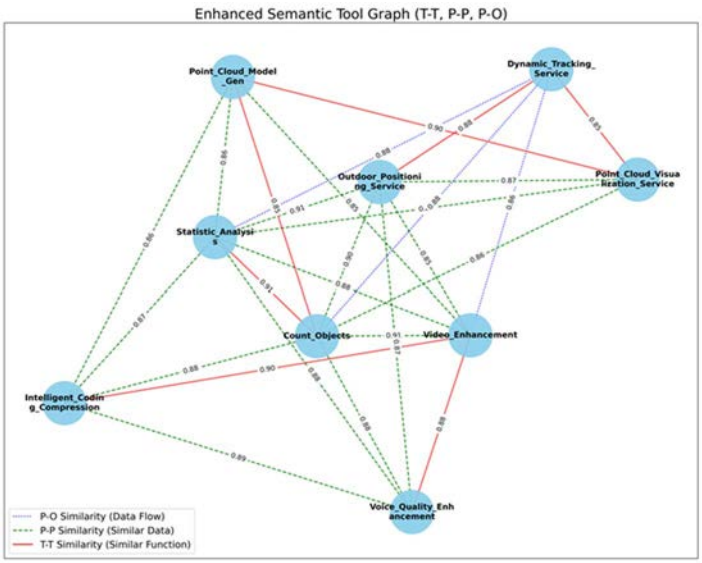


Figure A-1: An example of a SCG

The numerical value above each edge indicates the computed similarity score. Red solid lines, green dashed lines, and blue dashed lines represent P-O, P-P, and T-T similarities, respectively.

History

Version	Date	Status
V4.1.1	August 2026	Publication